GSoC 2022 Proposal

Social Street Smart

Akash Kumar

Table of contents

Creating Disinformation or Fact-checked feed and search React web application

- Technologies I plan to use
- Features I am planning to implement
- Plan of Action
- Detailed Implementation of routes -
  - Using Scrapy (a free and open-source web-crawling framework in Python) to crawl and scrape Fact-Checking Website.
  - Using Image-match (create image-signature) and Elasticsearch (search image-signature) to make fast and robust Reverse Image Search Engine

Test and Deploy the Project

- Reference Links

Timeline

- Community Bonding
- Week 1
- Week 2 and Week 3
- Week 4 and Week 5
- Week 6
- Mid Evaluations
- Week 7,8,9
- Week 10
- Final Week 11

My Past Contributions

Issues Opened:
MRs created:

My Projects :

Why are you the best person to execute this proposal? Prior Experience

Post GSoC plans

Personal Information

Name: Akash Kumar
University: Nalanda College Of Engineering, Chandi **Field of Study:** Computer Science and Engineering **Date of Enrollment:** August 2018
Expected Graduation date: August 2022
Degree: Bachelor of Technology
Year: 4th
Github: https://github.com/A7-4real
LinkedIn: www.linkedin.com/in/a7foreal
Gitter nick: @A7-4real
Email Address: a7.4real@gmail.com
colakashcruise7@gmail.com
Phone number: (+91) 7061059440
Timezone: Indian Standard Time (UTC +5:30)

Commitments

- **How many hours will you work per week on your GSoC project?**

  I am planning to spend 40 - 50 hours or more on the project per week.

- **What is the size of the project ?**

  Medium.

- **Other Commitments**

  I have no other commitments during the GSoC period.

- **Do you plan to apply for any other organisation for GSoC'22?**
  I am only applying to Aossie (Social Street Smart) for GSoC'22 and have no plans to contribute to any other organisation

- **If you're selected as a GSoC student, would you like to work on other tasks besides the projects of your choice?**
  Yes, I would love to work on other tasks that are not related to my GSoC project.

- **If you're not selected as a GSoC student, would you like to work on the projects as a general contributor?**

Yes, even if I'm not selected as a GSoC participant, I would happily continue working as a general contributor.

- **Would you like to contribute to Aossie in the long term, after the GSoC program ends?**
  Yes, I would like to contribute to Aossie even after GSoC ends.
- **What motivated you the most towards applying for GSoC?**
  There are various reasons for which I wanted to apply for GSoC but my main is working on an open source project which can have impact on our society. Recognized as a GSoC contributor is also a motivating factor. The stipend was not a motivating factor but an opportunity to work with a big organisation like Aossie was.

Introduction to the Project

With the advent of Internet, the problems faced by the people have also grown. These include abusive languages, fake news articles, click-baits, malicious websites and security attacks.

The aim of this project is to develop a Chrome Extension to make Internet a safer and more productive service for the users.

Social Street Smart chrome extension provides context menu for **Disinformation in Image** which enables user to lookup any Image on the web and get results about where that has been used.

Disinformation in Image (DII) at present -

- The chrome extension relied upon Google Cloud API for search results.
- It is the context menu in social street smart with title "Check image for disinformation" and "image" as a contexts
- A new popup chrome tab is created for results using "chrome.windows.create({})"
- The Disinformation in Images API requires you to generate a set of custom API keys for yourself.

Scopes of Improvement -

- User Experience of overall chrome extension can be improved by providing users the results overlays on the current tab. see here
- Including screenshot search context menu will provide more freedom and option to search image for disinformation in Image (eg - Sometimes users would just like to search part of Image. Therefore, using screenshot user can search any part of the webpage. see here
- Providing users an option to switch on hover image search button (overlay search button when user hover over image) will make searching DII fast and effortless . see here

- The DII API requires you to generate a set of custom API keys for yourself, which is extra burden.
- DII API uses Google Cloud Vision API under the hood, which is not free. An open source solution would make our DII API more scalable.
- The Image context (DII) results will be more relevant if we make fake-news specific Reverse image search engine.

My Primary Goals

- **Integrate Fact Checking Websites by crawling and scraping using scrapy :** As there are lot of fact-checking organizations are doing great job in debunking the disinformation Online. But the reach of these websites are limited and their content is not delivered to many people. We are going to crawl and scrape all the images and context related to them from the article in these fact-checking website. (Sub Project -1)

- **Building Reverse Image Search Engine (image-match + Elasticsearch) :** Building fast and robust Reverse Image Search Engine using image-match (make image signatures) and Elasticsearch (search image signature) from images and contexts related to them, which was scraped from fact-checking website. **(Sub Project - 1)**

- **Adding OCR support :** During my research on project I found that by extracting text from images helps in getting more context about image by making text extracted from the image searchable. eg - tweet. See here. **(Sub Project - 1)**

- **Improving our Social Street Smart Chrome Extension :** I would like to add these functionality in our SSS Chrome Extension this summer - **(Sub Project - 2)**
    1) Adding Screenshot search context menu. See here
    2) Same tab search result overlay. See here
    3) Hover image search button. See here

- **Creating latest fact-checked News feed and Search React App :** This react application will provide latest fact-checked feed in the world. It also would have Search component where search result populated through javascript, users can like, search and comment. We will overlay this react app on users current tab using content script when user search using our sss chrome extension (DII). see here. **(Sub Project - 3)**

- **Connecting our application using Rest API :** Our application will be talk to each other using Rest API build with Flask. **(Sub Project - 3)**

Architecture Diagram -

Why Social Street Smart (Disinformation in Image) ?

- With increase in advent Internet user there is increase in misinformation spreading which leads to lot of chaos and unpeacefulness in our world and community. This worries me.
- SSS takes step to reduce this. By contributing in this project I want to be part of this community. As most misinformation spread Online is in the form of posts, images, screenshots etc. Therefore DII got my attention
- Also the tech stack user in the project are one in which I have lot of experience and interest. I also have worked on this kind of project previously.

Therefore, Social Street Smart (DII) is the intersection of my core values and tech skills.

Implementation of the Project Goals

Integrating Fact Checking Websites by crawling and scraping using scrapy

Plan of Action

Technology I plan to use -

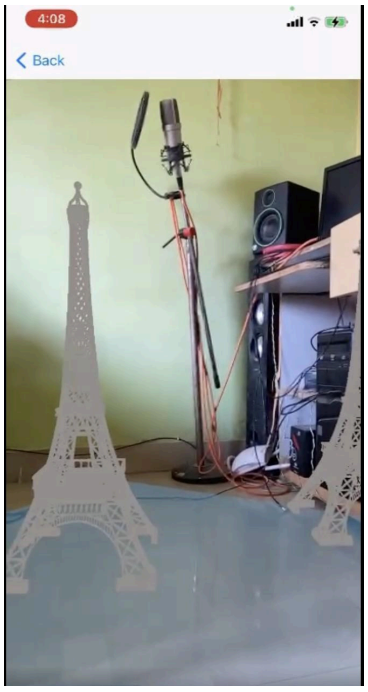
- Scrapy - Scrapy is a free and open-source web-crawling framework written in Python.

Why scrapy -

- Scrapy process can be used to extract the data from sources such as web pages using the spiders.
- Scrapy enables you to easily post-process any data you find. Data on the web is a mess. It is very unlikely that the data we find in fact-checking websites will be in the exact format that we would like it to be. Scrapy uses Item class to produce the output whose objects are used to gather the scraped data.
- Item Loaders provide a convenient mechanism for populating scraped items.
- Item Pipeline receives an item and performs an action over it, also deciding if the item should continue through the pipeline or be dropped and no longer processed.
- Scrapy can be used in selecting dynamically-loaded content.

Implementation Summary -

- Setup our scrapy project : We would first setup our Scrapy project.
- Analyze the layout and structure of selected fact-checking website : Carefully select the css selector (using browser developer console). Later we would use these selectors to parse the html response using our spider to extract the structured data we want.
- Coding our spider, item, item Loader and Item Pipeline : Our Spider will get unstructured data from the fact-checking website (eg - boomlive.in) , convert that unstructured data into structured data using **item** and **item loader** and add these structured data into our **elasticsearch** backend using **item pipeline**.



Rendered 3D model will look something like this.

.

Reference Links

https://flutter.dev/docs/development/add-to-app/ios/project-setup
h ttps://flutter.dev/docs/development/add-to-app/ios/add-flutter-screen?tab=engine-swift-tab h
ttps://www.appcoda.com/arkit-horizontal-plane/

https://github.com/Alamofire/Alamofire

Detailed Implementation

The implementation will include three parts.

1. Setting up our Scrapy project using scrapy
2. Analyze the layout and structure of selected fact-checking website
3. Coding our spider, item, item Loader and Item Pipeline

Setting up our Scrapy Project

we will first setup our new scrapy project, setting up scrapy project is fast and easy

```
(base) C:\Users\colak\Desktop>scrapy startproject DiiProject
New Scrapy project 'DiiProject', using template directory 'C:\Users\colak\miniconda3\lib\site-packages\scrapy\templates\
project', created in:
    C:\Users\colak\Desktop\DiiProject

You can start your first spider with:
    cd DiiProject
    scrapy genspider example example.com
```

This will create a DiiProject directory with the following content :

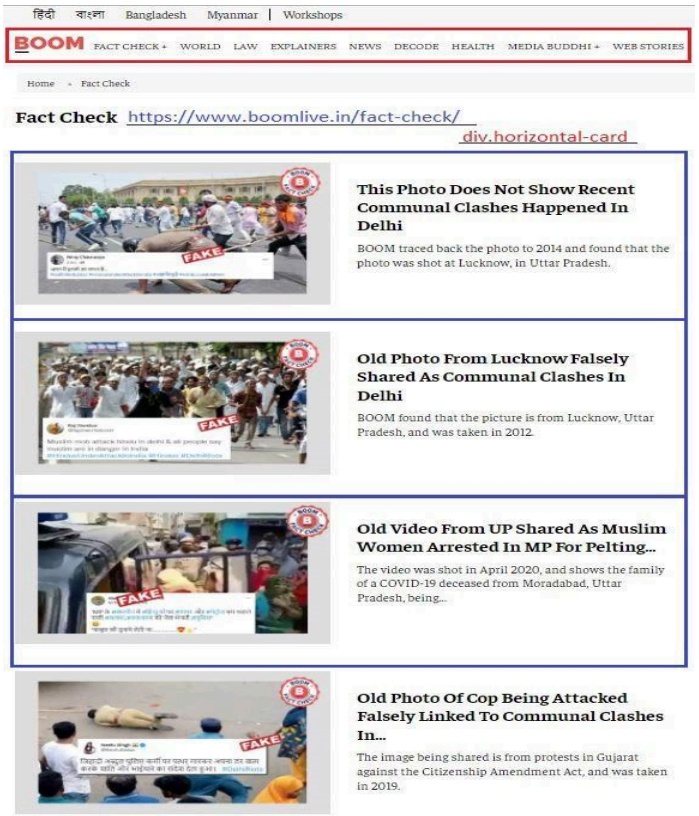
```
DiiProject/
    scrapy.cfg            # deploy configuration file

    DiiProject/             # project's Python module, you'll import your code from here
        __init__.py

        items.py          # project items definition file

        middlewares.py    # project middlewares file

        pipelines.py      # project pipelines file

        settings.py       # project settings file

        spiders/          # a directory where you'll later put your spiders
            __init__.py
```

Analyze the layout and structure of selected fact-checking website

Analyzing the layout and structure of fact-checking website is very important to get the right data for our use case.
For the proposal I am only showing to crawl and scarpy boomlive.in (fact-checking website) but I am planning to crawl and scrape contents from the following -

1) Alt News
2) WebQoof
3) PolitiFact
4) FactCheck.org
5) Factly

Boomlive website look something like this:



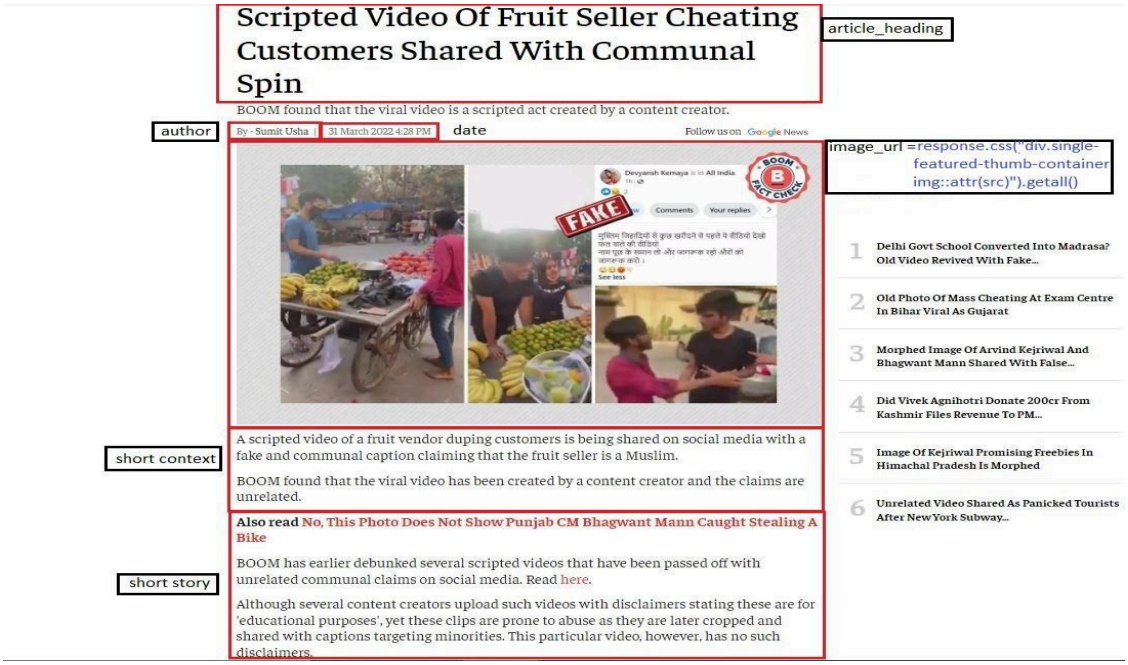
fact checked from different categories of news

each categories have many articles.

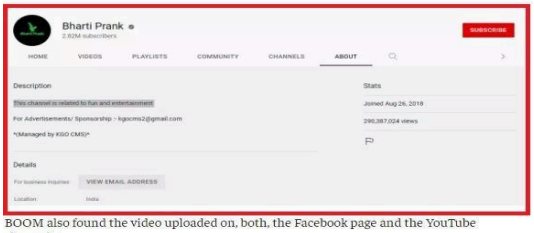
css selector - div.horizontal-card

Our scrapy spider will crawl all the articles from every category in Boom Website

Single article in boomlive.in look something like this -



BOOM also received the video on its helpline number with a similar claim.



BOOM also found the video uploaded on, both, the Facebook page and the YouTube channel.

Also read Old Video Of Doctor Suffering Heart Attack In Bengaluru Shared As Mumbai

The same video was uploaded on Raju Bharti's Facebook page on March 26, 2022 without any communal claim. The caption with the video reads '*United state*'. Several captions with the video suggest that it is a scripted act.

- As you can see the above example, the misinformation spread through different forms - whatsapp forwards , facebook posts, articles etc
- And each misinformation (eg - scripted video of fruit cheating customer with communal spin) spread through different images (or posts) to different people, but the context of every images (or posts) is same. Fact-checking website do great job in collecting all the different viral posts related to single misinformation topic.
- Therefore, we can use this to get context about every different images (or posts) related to single misinformation. This would result in getting the right contexts about the image searched by our user.

Coding our spider, item, item Loader and Item Pipeline :

Spiders **(fcSpider)** are classes that you define and that Scrapy uses to scrape information from a website (or a group of websites). They must subclass Spider and define the initial requests to make, optionally how to follow links in the pages, and how to parse the downloaded page content to extract data.

```
class fcSpider(scrapy.Spider):
    # 1
    name = 'boomCrawler-v1'

    # 2
    def start_requests(self):
        urls = [
            'https://www.boomlive.in/fact-check/',
            'https://www.boomlive.in/world/',
            'https://www.boomlive.in/law/',
            'https://www.boomlive.in/news',
            'https://www.boomlive.in/explainers'

        ]
        for url in urls:
            yield scrapy.Request(url=url, callback=self.parse)

    # 3
    def parse(self, response):
        allHorizontalDivs = response.css("div.horizontal-card")
        article_urls = []
        base_url = "https://www.boomlive.in"

        for div in allHorizontalDivs:
            url = div.css("a").attrib["href"]
            article_urls.append(url)

        for url in article_urls:
            url_modified = base_url + url
            yield scrapy.Request(url_modified, callback=self.parseArticle)
```

```
# 4
def parseArticle(self, response):

    # initializing our Item Loader of item Metadata
    metadata = ItemLoader(item=Metadata(), response=response)

    metadata.add_value("news_src", "boomlive.in")
    # images from first selector
    metadata.add_value("images_url",
                       response.css("div.single-featured-thumb-container img::attr(src)").getall())
    # images from second selector
    metadata.add_value("images_url",
                       response.css("div.image-and-caption-wrapper img::attr(src)").getall())
    # images from social media embeddings selectors
  # l.add_css("images_url", "youtube-thumbnail, twitter-embeds, instagram-embeds")
    metadata.add_value("src_article_url", response.url)
    metadata.add_value("article_heading", response.css(
        "header h1.is-custom-title::text").get())
    metadata.add_value("short_context", response.css("div h2::text").get())
    metadata.add_value("short_intro", response.css("div p::text").get())
    metadata.add_value("author", response.css(
        "div a.icon-link::text").get())
    metadata.add_value("date", response.css(
        "span span.convert-to-localtime::text").get())

    # initializing our Item Loader of item Image_context
    # image_context = ItemLoader(item=Image_context())
    # image_context_list = ItemLoader(item=DiiprojectItem)

    metadata_dic = metadata.load_item()
```

Walking through the code line by line.

1. Identifies the Spider "boomCrawler-v1". Unique within a project, you can't set the same name for different Spiders.
2. Must return an iterable of Requests (return a list of requests) which the Spider will begin to crawl from. Subsequent requests will be generated successively from these initial requests
3. a method that will be called to handle the response downloaded for each of the requests made. Extracting all the div.horizontal-card (all articles) in each page and calling parseArticle function with modified response
4. ParseArticle function will scrape all the required data. I am using item and item loader to extract structured data.

```python
from scrapy.item import Item, Field

# Metadata class


class Metadata(Item):

    # image context metadata
    news_src = Field(serializer=str)
    src_article_url = Field(serializer=str)
    # image url collected from different css selectors stored in python dict
    images_url = Field(serializer=dict)
    article_heading = Field(serializer=str)
    short_context = Field(serializer=str)
    short_intro = Field(serializer=str)
    author = Field(serializer=str)
    date = Field(serializer=str)
```

Item subclasses are declared using a simple class definition syntax and Field objects. Metadata class will defined all the required field in item.py of our project

```python
# useful for handling different item types with a single interface
from itemadapter import ItemAdapter
from scrapy.exceptions import DropItem


class DuplicatesPipeline:

    def __init__(self):
        self.ids_seen = set()

    def process_item(self, item, spider):
        adapter = ItemAdapter(item)
        if adapter['id'] in self.ids_seen:
            raise DropItem(f"Duplicate item found: {item!r}")
        else:
            self.ids_seen.add(adapter['id'])
            return item
```

"DuplicatesPipeline" will filter the duplicates while scraping according to the id

Item Pipeline could also be used to directly add these extracted data in our elasticsearch


```
{'article_heading': ['Hindu, Muslim Families Of Accused In Jahangirpuri Say '
                     "'False Charges'"],
 'author': ['Zafar Aafaq'],
 'date': ['18 April 2022 9:31 AM GMT'],
 'images_url': ['https://www.boomlive.in/h-upload/2022/04/18/975177-jahangirpuri-violence-what-really-happened.webp',
                'https://www.boomlive.in/h-upload/2022/04/18/975173-jahangirpuri.webp',
                'https://www.boomlive.in/h-upload/2022/04/18/975174-jahangirpuri-2.webp',
                'https://www.boomlive.in/h-upload/2022/04/18/975175-asfiya.webp',
                'https://www.boomlive.in/h-upload/2022/04/18/975176-jahangirpuri-3.webp'],
 'news_src': ['boomlive.in'],
 'short_context': ['Over 20 people have been arrested in Jahangirpuri '
                   'following a Hanuman Jayanti rally. Swords and firearms '
                   'have been recovered. What really led to the violence?'],
 'short_intro': [" A day after communal clashes broke out in Northwest Delhi's "
                 "Jahangirpuri, there's an eerie calm in the neighbourhood. On "
                 'Saturday evening, a crowd of Hindu revellers conducted a '
                 'rally – a '],
 'src_article_url': ['https://www.boomlive.in/news/jahangirpuri-violence-hanuman-jayanti-shobha-yatra-hindu-muslim-17585']}
```


scraped data from single article by our spider will look something like this

Building Reverse Image Search Engine (image-match + Elasticsearch)

Plan of Action

Technology I plan to use -

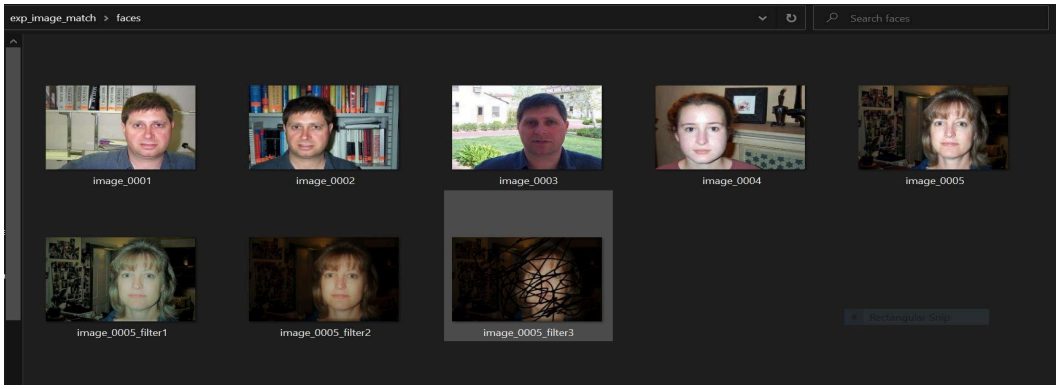
image-match - It is a simple open source package for finding approximate image matches from a corpus. It is similar, for instance, to pHash.
Elasticsearch - Elasticsearch is a search engine based on the Lucene library. It provides a distributed, multitenant-capable full-text search engine with an HTTP web interface and schema-free JSON documents

Why image-match and elasticsearch

- This algorithm is intended to find nearly duplicate images.
- Includes database backend that easily scales to billions of images and supports sustained high rates of image insertion: up to 10,000 images/s on our cluster.

Testing image-match -

I run test on image-match to check whether it fits our use case. I added image dataset (which is converted to image-signature by image-match and stored and searched through elasticsearch) and search the distorted version of the same image.



```
>>>
>>> ses.search_image("file:///C:/Users/colak/Desktop/exp_image_match/faces/image_0005_filter3.jpg")
    ses.search_image("file:///C:/Users/colak/Desktop/exp_image_match/faces/image_0005_filter3.jpg")
[{'id': 'G94Us38B40FMG9ofrktu', 'score': 4.0, 'metadata': {'things': 'women2'}, 'path': 'file:///C:/Users/colak/Desktop/exp_image_match/faces/image_0005.jp
g', 'dist': 0.3812354477985448}, {'id': 'D94Qs38B40FMG9ofC0tH', 'score': 1.0, 'metadata': {'things': 'car3'}, 'path': 'file:///C:/Users/colak/Desktop/exp_i
mage_match/car_side/image_0003.jpg', 'dist': 0.6911871496071536}]
>>>
>>>
```

```
>>>
>>> ses.add_image("file:///C:/Users/colak/Desktop/exp_image_match/faces/image_0005_filter1.jpg")
>>> ses.search_image("file:///C:/Users/colak/Desktop/exp_image_match/faces/image_0005_filter3.jpg")
[{'id': 'G94Us38B40FMG9ofrktu', 'score': 4.0, 'metadata': {'things': 'women2'}, 'path': 'file:///C:/Users/colak/Desktop/exp_image_match/faces/image_0005.jp
g', 'dist': 0.3812354477985448}, {'id': 'JN5Js38B40FMG9ofPEtY', 'score': 5.0, 'metadata': None, 'path': 'file:///C:/Users/colak/Desktop/exp_image_match/fac
es/image_0005_filter1.jpg', 'dist': 0.3918299349661063}, {'id': 'D94Qs38B40FMG9ofC0tH', 'score': 1.0, 'metadata': {'things': 'car3'}, 'path': 'file:///C:/U
sers/colak/Desktop/exp_image_match/car_side/image_0003.jpg', 'dist': 0.6911871496071536}]
>>>
```

```
>>>
>>> ses.add_image("file:///C:/Users/colak/Desktop/exp_image_match/faces/image_0005_filter2.jpg", metadata={"things":"women 2 with filter 2"})
>>> ses.search_image("file:///C:/Users/colak/Desktop/exp_image_match/faces/image_0005_filter3.jpg")
[{'id': 'G94Us38B40FMG9ofrktu', 'score': 4.0, 'metadata': {'things': 'women2'}, 'path': 'file:///C:/Users/colak/Desktop/exp_image_match/faces/image_0005.jpg
', 'dist': 0.3812354477985448}, {'id': 'JN5Js38B40FMG9ofPEtY', 'score': 5.0, 'metadata': None, 'path': 'file:///C:/Users/colak/Desktop/exp_image_match/faces
/image_0005_filter1.jpg', 'dist': 0.3918299349661063}, {'id': 'Jt5Ns38B40FMG9ofDktY', 'score': 5.0, 'metadata': {'things': 'women 2 with filter 2'}, 'path':
 'file:///C:/Users/colak/Desktop/exp_image_match/faces/image_0005_filter2.jpg', 'dist': 0.39900909966804043}, {'id': 'Jd5Ms38B40FMG9ofh0s_', 'score': 5.0, '
metadata': {'things': 'women 2 with filter 2'}, 'path': 'file:///C:/Users/colak/Desktop/exp_image_match/faces/image_0005_filter2.jpg', 'dist': 0.39900909966
804043}, {'id': 'D94Qs38B40FMG9ofC0tH', 'score': 1.0, 'metadata': {'things': 'car3'}, 'path': 'file:///C:/Users/colak/Desktop/exp_image_match/car_side/image
_0003.jpg', 'dist': 0.6911871496071536}]
>>>
```

Summary

- Setting up image-match and Elasticsearch : We would first setup our image-match and Elasticsearch.
- Understanding image-match : Understanding how image-match works
- Image-signatures and output format

Detailed Implementation

The implementation will include three parts.

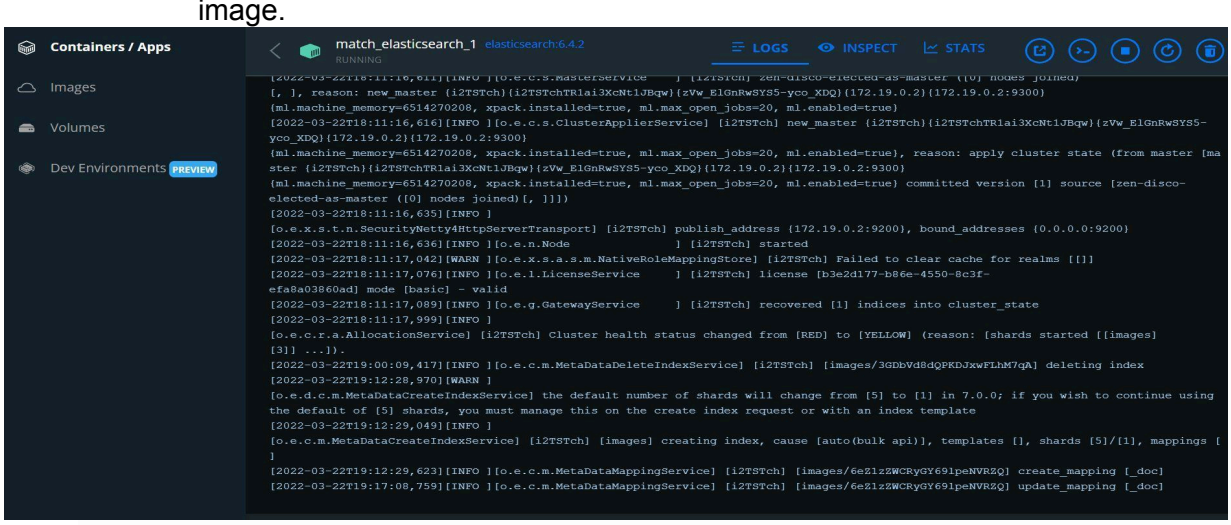
setting up image-match and Elasticsearch

setting up image-match

- Install image-match with pip
  $ pip install numpy
  $ pip install scipy
  $ pip install image_match
- Build from source
  Clone this repository: $ git clone https://github.com/ascribe/image-match.git
  Install image_match from the project directory:
  $ pip install numpy
  $ pip install scipy
  $ pip install .

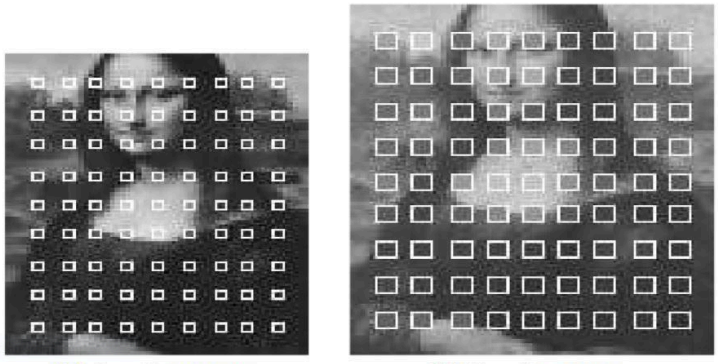
setting up Elasticsearch

- Elasticsearch can be installed and configured directly on operating system.
- Setting up by using Elasticsearch docker image - I using Elasticsearch docker image.



Understanding image-match

- Step 1 - Loads an image and convert to greyscale
- Step 2 - Crops an image, removing featureless border regions Computes grid points for image analysis
- Step 3 - Computes array of greyness means
- Step 4 -Computes differences in greylevels for neighboring grid points
- The flattened version of this array is the image signature.
- Step 5 - Flatten the array and return the signature



grid squres for mona lisa images

```
{'path': 'https://pixabay.com/static/uploads/photo/2012/11/28/08/56/mona-lisa-67506_960_720.jpg',

 # Image signature

 'signature': [0, 0, 0, 0, 0, 0, 2, 2, 0, 0, 0, 0, 0, 2, 2, 2, 0, 0, 0, 0, 2, 2, 2, 2, 0 ... ]
 'simple_word_0': 42252475,
 'simple_word_1': 23885671,
 'simple_word_10': 9967839,
 'simple_word_11': 4257902,
 'simple_word_12': 28651959,
 'simple_word_13': 33773597,
 'simple_word_14': 39331441,
 'simple_word_15': 39327300,
 'simple_word_16': 11337345,
 'simple_word_17': 9571961,
 'simple_word_18': 28697868,
 'simple_word_19': 14834907,
 'simple_word_2': 7434746,
 'simple_word_20': 37985525,
 'simple_word_21': 10753207,
 'simple_word_22': 9566120,
 ...
 'metadata': {'category': 'art'},
 }
```

```
>>> from elasticsearch import Elasticsearch
>>> from image_match.elasticsearch_driver import SignatureES
>>> es = Elasticsearch()
>>> ses = SignatureES(es)
>>> ses.add_image('https://upload.wikimedia.org/wikipedia/commons/thumb/e/ec/Mona_Lisa,_by_Leonardo_da_Vinci,_from_C2RMF_retouched.jpg/687
>>> ses.search_image('https://upload.wikimedia.org/wikipedia/commons/thumb/e/ec/Mona_Lisa,_by_Leonardo_da_Vinci,_from_C2RMF_retouched.jpg/
[
 {'dist': 0.0,
  'id': u'AVM37nMg0osmmAxpPvx6',
  'path': u'https://upload.wikimedia.org/wikipedia/commons/thumb/e/ec/Mona_Lisa,_by_Leonardo_da_Vinci,_from_C2RMF_retouched.jpg/687px-Mona
  'score': 0.28797293}
]
```

```
>>>
>>> from elasticsearch import Elasticsearch
>>>
>>> # Elasticsearch client
>>> client = Elasticsearch("http://localhost:9200")
>>>
>>>
>>> # get Index
>>> client.indices.get(index="*")
{'favorite_candy': {'aliases': {}, 'mappings': {}, 'settings': {'index': {'creation_date': '1649490745013', 'number_of_shards': '5', 'number_of_replicas': '1', 'u
uid': 'ZFVGV0eNTouxidELLX84gQ', 'version': {'created': '6040299'}, 'provided_name': 'favorite_candy'}}}, 'images': {'aliases': {}, 'mappings': {'_doc': {'properti
es': {'image': {'properties': {'metadata': {'properties': {'things': {'type': 'text', 'fields': {'keyword': {'type': 'keyword', 'ignore_above': 256}}}}}, 'path':
{'type': 'text', 'fields': {'keyword': {'type': 'keyword', 'ignore_above': 256}}}, 'signature': {'type': 'long'}, 'simple_word_0': {'type': 'long'}, 'simple_word_
1': {'type': 'long'}, 'simple_word_10': {'type': 'long'}, 'simple_word_11': {'type': 'long'}, 'simple_word_12': {'type': 'long'}, 'simple_word_13': {'type': 'long
'}, 'simple_word_14': {'type': 'long'}, 'simple_word_15': {'type': 'long'}, 'simple_word_16': {'type': 'long'}, 'simple_word_17': {'type': 'long'}, 'simple_word_1
8': {'type': 'long'}, 'simple_word_19': {'type': 'long'}, 'simple_word_2': {'type': 'long'}, 'simple_word_20': {'type': 'long'}, 'simple_word_21': {'type': 'long'
}, 'simple_word_22': {'type': 'long'}, 'simple_word_23': {'type': 'long'}, 'simple_word_24': {'type': 'long'}, 'simple_word_25': {'type': 'long'}, 'simple_word_26
': {'type': 'long'}, 'simple_word_27': {'type': 'long'}, 'simple_word_28': {'type': 'long'}, 'simple_word_29': {'type': 'long'}, 'simple_word_3': {'type': 'long'}
, 'simple_word_30': {'type': 'long'}, 'simple_word_31': {'type': 'long'}, 'simple_word_32': {'type': 'long'}, 'simple_word_33': {'type': 'long'}, 'simple_word_34'
: {'type': 'long'}, 'simple_word_35': {'type': 'long'}, 'simple_word_36': {'type': 'long'}, 'simple_word_37': {'type': 'long'}, 'simple_word_38': {'type': 'long'}
, 'simple_word_39': {'type': 'long'}, 'simple_word_4': {'type': 'long'}, 'simple_word_40': {'type': 'long'}, 'simple_word_41': {'type': 'long'}, 'simple_word_42':
 {'type': 'long'}, 'simple_word_43': {'type': 'long'}, 'simple_word_44': {'type': 'long'}, 'simple_word_45': {'type': 'long'}, 'simple_word_46': {'type': 'long'},
 'simple_word_47': {'type': 'long'}, 'simple_word_48': {'type': 'long'}, 'simple_word_49': {'type': 'long'}, 'simple_word_5': {'type': 'long'}, 'simple_word_50':
{'type': 'long'}, 'simple_word_51': {'type': 'long'}, 'simple_word_52': {'type': 'long'}, 'simple_word_53': {'type': 'long'}, 'simple_word_54': {'type': 'long'},
'simple_word_55': {'type': 'long'}, 'simple_word_56': {'type': 'long'}, 'simple_word_57': {'type': 'long'}, 'simple_word_58': {'type': 'long'}, 'simple_word_59':
{'type': 'long'}, 'simple_word_6': {'type': 'long'}, 'simple_word_60': {'type': 'long'}, 'simple_word_61': {'type': 'long'}, 'simple_word_62': {'type': 'long'}, '
simple_word_7': {'type': 'long'}, 'simple_word_8': {'type': 'long'}, 'simple_word_9': {'type': 'long'}, 'timestamp': {'type': 'date'}}}}}, 'settings': {'index':
{'creation_date': '1647976348971', 'number_of_shards': '5', 'number_of_replicas': '1', 'uuid': '6eZ1zZWCRyGY69lpeNVRZQ', 'version': {'created': '6040299'}, 'provi
ded_name': 'images'}}}}
>>>
```

Index (image-signatures) stored in my elasticsearch

Adding OCR Tesseract support

Technology I Plan to use

- Tesseract OCR - Tesseract is an optical character recognition engine for various operating systems.
- Tesseract ocr is free and could easily be installed on any operating system

Features I am planning to implement

- By extracting texts from image give us more option to search.
- Extracted text from the image could be used to search in indexed document later


```
print(pytesseract.image_to_string(Image.open("C:/Users/colak/Desktop/974058-putin-02-ocr-test-2.jpg")))
```

```
Pakistan is just a Cemetery for
Pakistanis

Russian President

Hyder Abbasi
15 March at 22:18-@

RUSSIAN PRESIDENT, Vladimir Putin Says:
Pakistan Is A Cemetery For Pakistani's:

"When a Pakistani becomes rich, his bank
accounts are in Switzerland. He travels to
London/ America for Medical treatment.

He invests in Uk/ Dubai. He buys from Dubai. He
consumes Chinese. He prays in Rome or Mecca.
His children study in Europe. He travels to
Canada, USA, Europe for tourism.

If he die, he will be buried in his native country of
Pakistan.

Pakistan is just a cemetery for Pakistani's. How
could a cemetery be developed?" See less
```

The text extracted from image by ocr could be used to make our search result faster and more reliable

Improving Social Street Smart chrome Extension

Plan of Action

Features I am planning to Implement

- Screenshot search context menu
- Current tab result overlay
- Hover image search button

Screenshot search context menu

Implementation summary

- Screenshot search context menu will provide users an option to Reverse Image search even if it is not image.
- Sometimes users want to only search just part of the image. This could be implement using **chrome.tabs.captureVisibleTab API** that could be used in **background script**
- Chrome.tabs.captureVisibleTab API takes screenshot of full visible Tab. But we don't want.
- We would allow user to select the area they want to take screenshot by injecting **canvas** on users current tab, this could be done using **content script.**
- Since content scripts run in the context of a web page and not the extension, they often need some way of communicating with the rest of the extension.
- The communication between background sript and content script is done via **Message passing**
- screenshot url is in base64 we need to convert it into bytestream for search through our reverse image search engine

Detailed Implementation -

Implementation is done in following step -

step 1 - Creating Screenshot context menu
step 2 - Activate screenshot search extension when context menu id of same as screenshot search context menu id is clicked
step 3 - Take screenshot using chrome.tabs.captureVisibleTab API in background script and send screenshot url as a message to content script
step 4 - Insert canvas and allow user to select area for screenshot
step 5 - converting base64 image to bytestream

step 1 - defining our context menu having name = "SSS-screenshot" and contexts : ["all"] which which allow extension to work on any contexts. Create "SSS-screenshot" context menu using contextMenuItem

```
// creating screenshot context menu
var contextMenuItem = {
  id: "s",
  title: "SSS-Screenshot",
  contexts: ["all"],
};

chrome.contextMenus.create(contextMenuItem);
console.log("context item created");
```

step 2 - **chrome.contextMenus.onClicked.addListener** listens when user click on context menu and check whether context id is same as our context menu item id

step 3 - - we need take screenshot of current and active tab only, therefore would check using **chrome.tabs.query**.
- Then we would capture screenshot of full visible tab using **chrome.tabs.captureVisibleTab** and save the url of screenshot using var screenshot = dataUrl
- will send message and screenshot Url to content script using **chrome.tabs.sendMessage**

```
// addListener listens when user clicks on context menu and check whether id is
// same as our menuItemId
chrome.contextMenus.onClicked.addListener(function (clickData) {
  if (clickData.menuItemId == "s") {
    console.log("context item clicked");

    // we to take screenshot of current and active tab
    chrome.tabs.query({ active: true, currentWindow: true }, function (tabs) {
      // Capture screenshot of full visible tab
      chrome.tabs.captureVisibleTab(null, null, function (dataUrl) {
        console.log("Tab screenshot initiated");

        // screenshot url
        var screenshotUrl = dataUrl;
        console.log("screenshot url - ", screenshotUrl);

        // send Message to content to content script with screenshotUrl as data
        chrome.tabs.sendMessage(
          tabs[0].id,
          { action: "take_screenshot", data: screenshotUrl },
          function (response) {
            console.log("screenshot done!");
          }
        );
      });
    });
  }
});
```

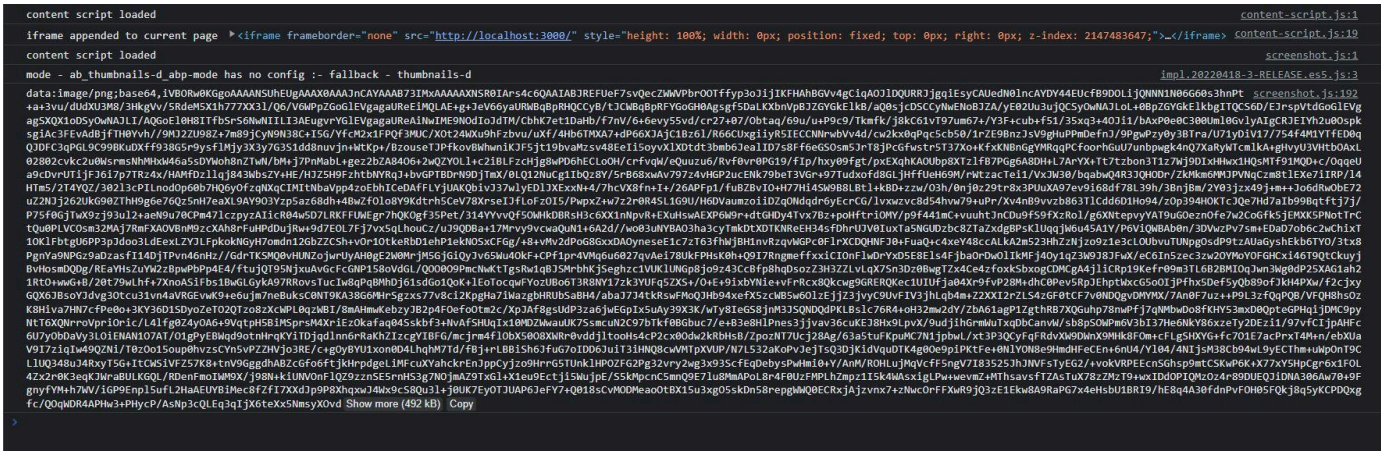
step 4 - In content script chrome.extension.onMessage listens for message containing message (msg.action and msg.action) from background script.
Then we will append required html and css to users web page using content script to select the desired area using **moveCover** and **on mouseup or mousedown** eventListener of jquery function and take screenshot

```
// addListener listens for the msg containing action and screenshot url
chrome.extension.onMessage.addListener(function (msg, sender, sendResponse) {
  if (msg.action == "take_screenshot") {
    // console.log("Message recieved from eventpage");
    var screenshotUrl = msg.data;
    // console.log("screenshot url - ", screenshotUrl);
    // // takeScreenshot(screenshotUrl);

    // append style to users page
    $(document.head).append(
      "<style>#SSS-screenshot{margin:0px;border:0px;padding:0px;border-radius:0px}#SSS-screenshot *{transition:none;margin:0px;border:0px;padding:0px;border-radius:0px}</style>"
    );

    var div = $(
      '<div id="SSS-screenshot" style="width:100%;height:100%;z-index:9999999999999999999;position:fixed;left:0px;top:0px;" ></div>'
    );

    // creating image object
    var img = new Image();

    // set image src attribute same as screenshotUrl
    img.src = screenshotUrl;

    var defaultCSS = "margin:0;padding:0;position:absolute;";

    // append div when image loads
    img.onload = function () {
      div.append(
        '<div class="SSS-screenshot-search" style="' +
          defaultCSS +
          'cursor:pointer;box-sizing:content-box;height:29px;width:29px;position:absolute;z-index:3;float:right;background-color:rgba(130, 186, 255, 0);"><svg viewBox="0 0 300 300" width="300" height="300" xmlns="http://www.w3
      );

      a; // append canvas to users web page
      div.append(
        "<canvas width=" +
          img.width +
          " height=" +
          img.height +
          ' style="' +
          defaultCSS +
          'height:100%;width:100%;top:0;left:0;position:absolute" class="SSS-screenshot-canvas"></canvas>'
      );

      div.append(
        '<div class="SSS-screenshot-close" style="' +
          defaultCSS +
          'position:absolute;z-index:3;cursor:pointer;user-select: none;width: 29px;height: 29px;font-size: 30px;text-align: center;line-height:0px;background: rgba(193, 22, 59, 0);"><svg viewBox="0 0 300 300" width="300" heig
      );

      div.append(
        '<div class="SSS-screenshot-cover" style="' +
          defaultCSS +
          'pointer-events:none;position:absolute;left:0;top:0;background-color:rgba(0,0,0,0.418)"></div>'
      );
```

```
function moveCover(parent) {
  var canvasTop = parent.find(".SSS-screenshot-canvas").offset().top;
  var canvasLeft = parent.find(".SSS-screenshot-canvas").offset().left;
  var canvasWidth = parent.find(".SSS-screenshot-canvas").width();
  var canvasHeight = parent.find(".SSS-screenshot-canvas").height();
  var left = Math.min(left1, left2);
  var top = Math.min(top1, top2);
  var bottom = Math.max(top1, top2);
  var right = Math.max(left1, left2);
  var width = Math.abs(left1 - left2);
  var height = Math.abs(top1 - top2);

  parent.find(".SSS-screenshot-cover").css({
    top: top + halfBall + "px",
    left: left + halfBall + "px",
    width: right - left + "px",
    height: bottom - top + "px",
  });

  var buttonLeft = right + 2 * halfBall;
  var buttonTop = bottom + 2 * halfBall;
  var buttonOpacity = 1;

  if (canvasWidth - right < 80) {
    buttonOpacity = 0.5;
    buttonLeft -= 80;
  }

  if (canvasHeight - bottom < 46) {
    buttonOpacity = 0.5;
    buttonTop -= 40;
  }

  parent.find(".SSS-screenshot-search").css({
    top: buttonTop + "px",
    left: buttonLeft + "px",
    opacity: buttonOpacity === 1 ? 0.7 : 1,
  });

  parent.find(".SSS-screenshot-close").css({
    top: buttonTop + "px",
    left: buttonLeft + 37 + "px",
    opacity: buttonOpacity === 1 ? 0.7 : 1,
  });
}
```

step 5-

As screenshot url is base64 image we need to convert it to blob for our use case which could easily done using javascipt.

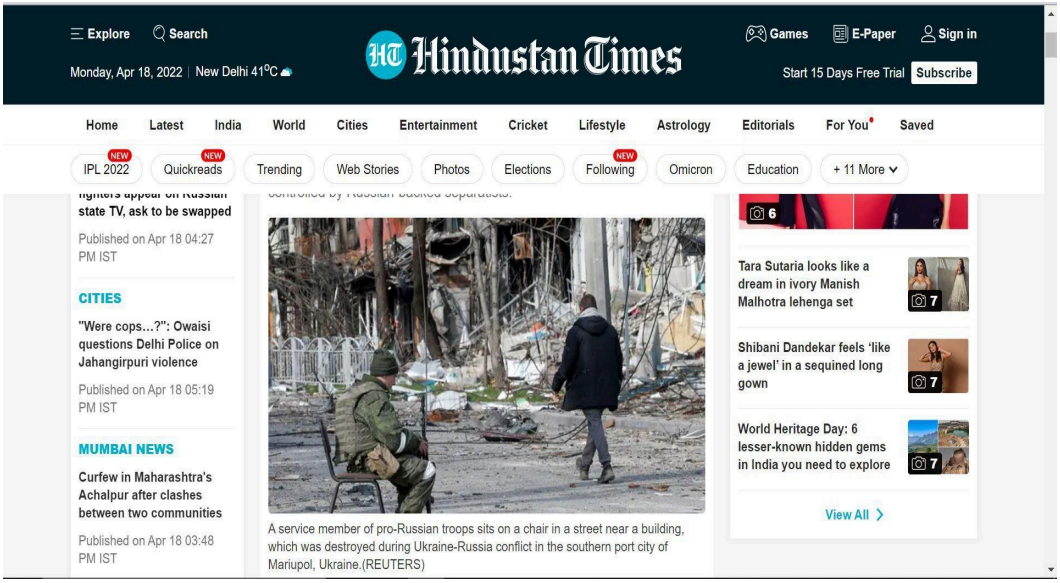


Note - due to space constrains could no paste whole screenshot logic. Please checkout screenshot chrome extension repository - link

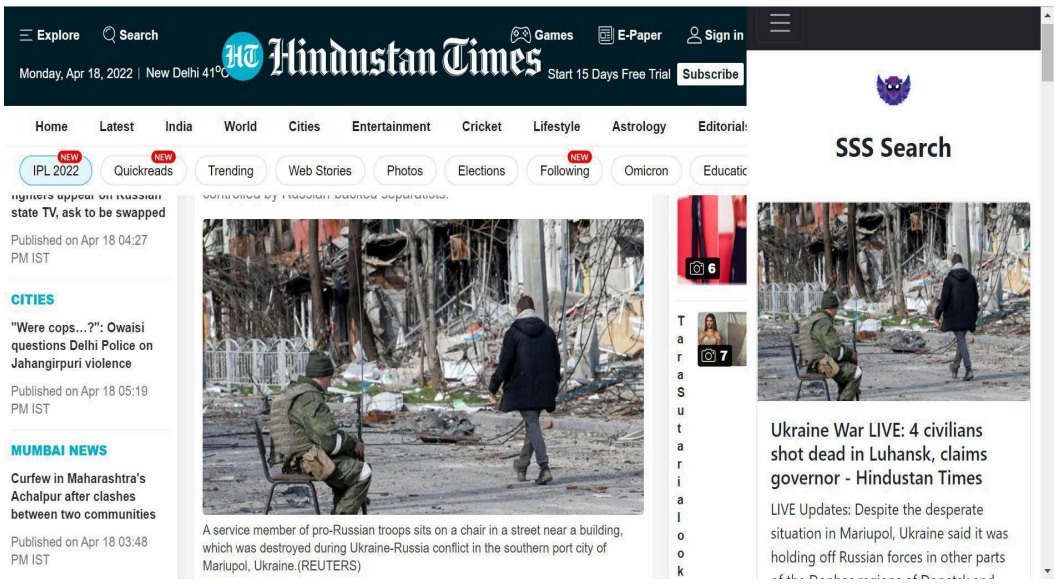
Current tab result overlay

Implementation summary

- User Experience of overall chrome extension can be improved by providing users the results overlays on the current tab.
- When user clicks our DII context menu **(background script)** to get the context about image, A right side result overlay (**react app**) will be created on the users current tab **(using content script)** displaying results fetched from the server .
- Our overlay will takes some width, so we will adjust users web page width using **window.innnerWidth.**
- Users will be provided a button to close the overlay
- This make our user experience great.



web page before result overlay



web page after result overlay

Detailed Implementation -

Implementation is done in following step -

step 1 - Creating Disinformation in Image context menu
step 2 - Activate Disinformation in Image extension when context menu id of same as screenshot search context menu id is clicked
step 3 - Send message from background script to content script to open search result on active and same tab.
step 4 - content script will insert iframe with src of our react app.
step 5 - Change the width of current tab and set width of our iframe overlay

```
console.log("eventPage Loaded");

// Defining our context menu with image as contexts
var contextMenuItem = {
  id: "RIS",
  title: "Disinformation in Image",
  contexts: ["image"],
};

// creating context menu with "Disinformation in Image" as title
chrome.contextMenus.create(contextMenuItem);
console.log("context item created");

// When user clicks a context menu we would check whether context menu id is
// same as "Disinformation in Image context menu" id
chrome.contextMenus.onClicked.addListener(function (clickData) {
  if (clickData.menuItemId == "RIS")
    console.log("Disinformation in Image context item clicked");

  // we would send message current and active tab
  chrome.tabs.query({ active: true, currentWindow: true }, function (tabs) {
    console.log("Now sending message to content script...");
    // send message to content script
    chrome.tabs.sendMessage(
      tabs[0].id,
      { action: "openSearchResult", data: clickData.srcUrl },
      function (response) {
        console.log("searchResult overlay opened!");
        console.log(response);
      }
    );
  });
});
```

our background script

```
console.log("content script loaded");

// iframe attributes
var body = document.getElementsByTagName("body")[0];
var iframe = document.createElement("iframe");
iframe.style.height = "100%";
iframe.style.width = "0px";
iframe.style.position = "fixed";
iframe.style.top = "0px";
iframe.style.right = "0px";
iframe.style.zIndex = "9000000000000000000";
// temporary react app as iframe src
iframe.src = "";
// console.log("iframe created", iframe.src);
// append iframe with 0 width
document.body.appendChild(iframe);
console.log("iframe appended to current page", iframe);

// change width of iframe to 370px
function toggle() {
  // predefined iframe width
  let iframeWidth = 370;

  // change width if toggle function is called and iframe Width is 0
  if (iframe.style.width == "0px") {
    // get current window width
    let currentWidth = window.innerWidth;
    let setWidth = currentWidth - iframeWidth;
    document.body.style.width = `${setWidth}px`;
    iframe.style.width = `${iframeWidth}px`;
  } else {
    document.body.style.width = `${currentWidth}px`;
    iframe.style.width = "0px";
  }
}
chrome.runtime.onMessage.addListener(function (msg, sender, sendResponse) {
  if (msg.action == "openSearchResult") {
    console.log("Message recieved", msg.action);
    var image_url = msg.srcUrl;
    // changing iframe src to our react app which will fetch result from our backend using react router (URL parameters)
    iframe.src = "http://localhost:3000/image_url";
    toggle();
    console.log("toggled function called");
  }
  sendResponse(true);
});
```

our content script

Hover Image search button

Implementation summary

- Providing users an option to switch on hover image search button (overlay search button when user hover over image) will make searching DII fast and effortless .
- This could be done using content script of our chrome extension.
- We would loop through all the image of users html (web page) this could be done with help of content script as content script can access users web page.
- After getting all the images we would inject the button with display as none onto all the images.
- After injecting all the button we would use addEventListener to listen for the hover event, when user hover over the image change the display of button to block and change the display of the button to none when user hover away the image

```
var imagesArray = document.getElementsByTagName("img");
console.log(imagesArray);

for (const img of imagesArray) {
  var button = document.createElement("button");
  button.innerHTML = "button";
  button.onclick = function () {
    console.log("clicked");
  };

  img.append(button);
}
```

content script

Demo video -

Timeline

Community Bonding (May 20 - June 12)	• I'll get to know my mentors (@MiHarsh and @Thuva4). I'll discuss the best practices which relevant for the project. Get up to speed to begin working on the prject • I'll improve the current DII implementation by adding Screenshot image search functionality in our chrome extension • I'll constantly be adding desired features and fixing bugs mentioned in the Scope of Improvement section as much as I can. • I'll be learning about unit testing and deployments, because I don't have much experience writing tests . But I do know the basics. • I'll also be constantly learning about manifest v3, scrapy to improve my implementation of project
Week 1 (June 13 - June 19)	• Building first version of scrapy spider • setting up image-match and elasticsearch

Week 2 and Week 3 (June 20 - July 3)	• Scaling spider to more fact-checking website. • Create first version of fact-checked feed and search component of react application. • I'll implement build flask api serve and connect our frontend with backend.
Week 4 , 5, 6 (July 4 - July 24)	• I will start implementing remaining features of chrome extension - search result overly and image hover search button. • Till now i would finish up with react application
Week 7 (July 25 - July 31) Mid Evaluations	• At this point, I'll have delivered complete feed and search react application. • Till now i would finish up with complete implementation of DII chrome extension and promised features
Week 8 and week 9 (August 1 - August 14)	• I'll be continuously fixing issues and bugs. • I'll be constantly be in touch with my mentors for correct guidance and implementation of project • I'll add OCR support to DII.
Week 10 (August 15 - August 21)	• At this time, I will test the application on different devices and try to fix as many bugs as possible.

Week 11, 12 (August 22 - September 5)	● I'll write unit test and deploy the application on awl lambda with the help of my mentors and my peers
Final Week 13 (September 5 - September 12)	● I'll add documentation and will clean up the application and code for final submission.

My Past Contributions -

Pull requests Opened:
https://github.com/ProvenanceLabs/image-match/pull/155
https://github.com/camicroscope/docs/pull/1
https://github.com/camicroscope/caMicroscope/pull/556

Issue opened by me are listed

Issues Opened:
https://github.com/ProvenanceLabs/image-match/issues/156

Why are you the best person to execute this proposal?

I have a fair experience in building applications using Reactjs, APIs, python and javascript which is the required tech-stack for the project . I have worked on several javascript , react ,firebase, python and flask api projects. I do have the skill set and experience to execute the proposed work. Also worked really hard since two month in search and experimenting the right tools and tech stack for this project.

Apart from skillset , I am wnat to become a dedicated open source contributer, coming from college where open source contributions is not very much popular. I want to use GSoC 22 experience to spread awareness about the importance and need of open source contributions in organizations like AOSSIE and others. This would result in increase participation and enthusiasm in open source contributions.

Prior Experience

I have pretty much experience in developing javascript, reactjs and python projects. I have been building reactsjs and python application since last 2 years.

I have build full stack amazon clone using Reactjs, In this i have used react router, react state, hooks, firebase for hosting, authentication and stripe api for payment service.

More reactjs project include News feed website using News api, Netflix UI Clone, Whatsapp clone using react and web sockets

I have previously co-founded and worked as a software Developer in Bio-Medical startup named "UPCHAAR"

I have machine learning training from IIT Kanpur

Post GSoC plans

1) Spreading awarenes about Misinformation

2) Expanding to our DII API to auto detect manipulation in image.